

Stage 2 — Performance Analysis

After the first round of optimizations, here are the remaining bottlenecks found in the hot paths, ordered by estimated impact.

10. Cache dominant frequency index on Ray

Where: `calc_ray` (Ray.py:107), `v_ray` (objects_2d.py:80), `tl` (objects_2d.py:98)

Problem: Every `calc_ray` call computes `np.argmax(np.abs(f0))` on a 500-element complex array. This does 500 complex→float absolute values + a 500-element argmax scan. However, `fshift_dispersion` only **rotates phases** (`exp(-j...) * f`), which never changes the magnitudes. The dominant frequency bin is therefore *invariant* for the entire lifetime of a ray and all its descendants.

Fix: Compute the index once in `Ray.__init__`, store as `_dom_freq_idx`. Use it everywhere instead of recomputing.

Savings: Eliminates `np.abs(complex[500])` + `np.argmax(float[500])` from every single `calc_ray`, every `v_ray` fallback, and every `tl` fallback — i.e., every propagation step of every ray.

Estimated speedup: 1.2–1.5×

11. Pre-store dispersion phase coefficient on Ray

Where: `fshift_dispersion` (objects_2d.py:110–116), `signal_on_ray` (Sensors.py:135)

Problem: `fshift_dispersion` is called every `calc_ray`:

```
python

t_d = x / ray.fft_speed          # 500 float divisions
f_d = np.exp(-1j * 2 * np.pi * ray.fft_freq * t_d) * f # 500 multiply + 500 exp + 500 multiply
```

The coefficient `-2πj * fft_freq / fft_speed` is constant for a given ray (same direction, same medium). Both `fshift_dispersion` and `signal_on_ray` independently reconstruct it.

Fix: Pre-compute `_phase_coeff = -2j * np.pi * fft_freq / fft_speed` once in `Ray.__init__`. Then:

```
python

f_d = np.exp(self._phase_coeff * x) * f # 500 multiply + 500 exp + 500 multiply
```

Saves the 500-element division and an intermediate temporary array per step.

Estimated speedup: 1.1–1.3×

12. Batch 2D interpolation for `fft_speed`

Where: `Ray.__init__` (Ray.py:77–78), `load_ray` (ray_utils.py:361)

Problem: `fft_speed` computation loops 500 times over a Python lambda that calls the Numba `interp2d_regular_numba` kernel. Each call has:

- Python→Numba dispatch overhead
- Tuple creation `(fi * _th_factor, _theta)`
- Numba function entry/exit

For a fixed theta, the y-index `j` and interpolation weights `v, (1-v)` are identical across all 500 calls. Only the x-index (frequency) varies.

Fix: Write a new Numba function `interp2d_batch_x` that:

1. Finds the y-index once for the shared theta
2. Loops internally over the array of f values
3. Returns an array of results

This replaces 500 Python→Numba round-trips with 1.

Estimated speedup: 1.3–2× on Ray.**init** (the initial 4002 rays + all spawned rays)

13. Share `fft_freq` across rays

Where: `Ray.__init__` (Ray.py:69), `Beam.__init__` (Ray.py:453–468)

Problem: Every Ray computes `rfftfreq(len(t), d=(t[-1]-t[0])/len(t))[:len(freq)]` identically — same `t` vector, same `nfft`. With 4002 initial rays + hundreds of spawned rays, this is thousands of redundant scipy calls and array allocations.

Fix: Compute `fft_freq` once in `Beam.__init__` (it already has it at line 430). Pass it to `Ray.__init__` and assign by reference. For spawned rays in `mode_change`, copy from the parent ray.

Estimated speedup: 1.05–1.1× (small per-ray but × thousands of rays)

Summary — Recommended implementation order

#	Optimization	Impact	Complexity
10	Cache dominant freq index	1.2–1.5×	Low
11	Pre-store phase coefficient	1.1–1.3×	Low

#	Optimization	Impact	Complexity
12	Batch 2D interpolation	1.3–2× on init	Medium
13	Share <code>fft_freq</code>	1.05–1.1×	Low

Start with 10 and 11 — they're the cheapest changes with broad impact across both tracing and signal phases.